

Description “Commit1”

- เขียนรายละเอียดการทำงานของโค้ดเพิ่มเติม ใน function ดังนี้
 - `def draw_square (x,y,size, fill_color, stroke_color, corner_weight):`
 - `class Board:`
 - `def __init__(self, size, cell_size, origin_x, origin_y):`

```
def draw_square (x,y,size, fill_color, stroke_color, corner_weight):
```

แนวคิดหลักของฟังก์ชันนี้

ต้องวาดสี่เหลี่ยม 1 ช่อง ที่เหมือนเป็นพื้นทึบ ดังนั้นใช้หลักการวาดเส้นตรงแนวนอนให้เป็นสีพื้นแล้ววาดกรอบ
ทับทีหลัง ฟังก์ชันนี้เลยแบ่งงานออกเป็น 2 ส่วน

ส่วน 1 สร้างพื้นที่ ด้วยการวาดเส้นแนวนอนที่มีความหนาเท่ากับพื้นที่ทั้งหมด

ส่วน 2 วาดกรอบทับด้านบน ด้วยเส้นหนาตามที่กำหนด (สีต่างจากพื้น) ซึ่งแต่ละจุดต้องการความหนา
กรอบไม่เท่ากัน เช่น ช่องบนกระดาน และชิ้นส่วนในมือผู้เล่น

Pseudocode

```
def draw_square(x, y, size, fill_color, stroke_color, corner_weight):
```

===== ส่วนที่ 1: สร้างพื้นที่ด้วยเส้นหนาเส้นเดียว =====

ตั้งสีเส้น ให้เป็น fill color

ตั้งความหนาเส้น ให้เป็น size # ความหนา = เท่ากับความสูงตัวก่อน

วาดเส้นแนวนอน จาก $(x, y + \text{size}/2)$ ถึง $(x + \text{size}, y + \text{size}/2)$

==== ส่วนที่ 2: วาดกรอบทับด้านบน 4 เส้นตรงๆ ====

ตั้งสีเส้น ให้เป็น stroke color

ตั้งความหนาเส้น ให้เป็น corner weight

วาดเส้นขอบบน จาก (x, y) ถึง $(x + \text{size}, y)$

วาดเส้นขอบล่าง จาก $(x, y + \text{size})$ ถึง $(x + \text{size}, y + \text{size})$

วาดเส้นขอบซ้าย จาก (x, y) ถึง (x, y + size)

วาดเส้นขอบขาว จาก $(x + \text{size}, y)$ ถึง $(x + \text{size}, y + \text{size})$

```
class Board:
```

```
    def __init__(self, size, cell_size, origin_x, origin_y):
```

แนวคิดการทำงานหลัก การทำงานของฟังก์ชันนี้สามารถแบ่งออกเป็น 2 ส่วนหลัก

ส่วนที่ 1 เป็นการจับเก็บค่าพารามิเตอร์ที่รับเข้ามาไว้ใน Attributes ของออบเจกต์ (self.size, self.cell_size, self.ox, self.oy)

ส่วนที่ 2 เป็นการสร้างโครงสร้างข้อมูลกระดาน (self.grid) ในรูปแบบ 2D array โดยทุกช่องมีค่าเริ่มต้นเป็น 0 ซึ่งหมายถึงสถานะว่างทั้งหมด พร้อมให้ method อื่นในคลาสเดียวกัน นำไปใช้อ่านและสามารถแก้ไขสถานะของกระดานในลำดับต่อไป

Pseudocode

เมธอด __init__(self, size, cell_size, origin_x, origin_y):

=== ส่วนที่ 1: เก็บค่าพารามิเตอร์ ===

self.size = size

self.cell_size = cell_size

self.ox = origin_x

self.oy = origin_y

=== ส่วนที่ 2: สร้างกระดาน โดย fix แถวก่อน แล้วเติมค่า 0 ที่ละคอลัมน์ทีละหลัง ===

self.grid = list ว่างเปล่า

row_idx = 0

ทำซ้ำ ขณะที่ row_idx < self.size:

 เพิ่มแถวว่างใหม่ [] เข้าไปที่ท้าย self.grid

 col_idx = 0

 ทำซ้ำ ขณะที่ col_idx < self.size:

 เพิ่มค่า 0 เข้าไปในแถวที่ row_idx ของ self.grid

 col_idx = col_idx + 1

 row_idx = row_idx + 1

Description “Commit2”

- มีการเขียนการทำงานโค้ดเพิ่มเติม ใน function ต่อไปนี้

`Board.draw()` เพื่อวาด board เริ่มต้น

`setup()` เพื่อสร้าง object ของ class Board

`draw()` เพื่อเรียก `board.draw()` และกำหนดสี background

class Board :

def draw(self) :

แนวคิดการทำงานหลัก

การทำงานของฟังก์ชันนี้สามารถแบ่งออกเป็น 2 ส่วนหลัก

ส่วนที่ 1 ไล่ทุกช่องด้วยลูปซ้อนลูป พร้อมคำนวณตำแหน่งพิกเซลแบบบวกสะสม โดยใช้ nested loop ด้วยตัวแปร `row_idx` และ `col_idx` เพื่อไล่เข้าถึงทุกช่องบนกระดาน (`self.grid`) ให้ครบทุกตำแหน่ง พร้อมคำนวณตำแหน่งพิกเซลจริงบนหน้าจอ (`px, py`) ควบคู่กันไปในแต่ละรอบ ด้วยวิธี บวกสะสม ค่า `self.cell_size` ที่ละช่อง โดย `py` จะสะสมเพิ่มทุกครั้งที่จบแถว ส่วน `px` จะสะสมเพิ่มทุกครั้งที่ขึ้นคอลัมน์ใหม่ และต้องรีเซ็ตกลับเป็น `self.ox` ทุกครั้งที่ขึ้นแถวใหม่

ส่วนที่ 2 ตรวจสอบค่าในช่อง จากนั้นเลือกสีแล้ววาดช่องสี่เหลี่ยมที่ละช่องจนครบทุกช่องบนกระดาน โดยเริ่มจากตรวจสอบค่าที่เก็บอยู่ในแต่ละช่องของ `self.grid` ผ่านเงื่อนไข `if/else` เพื่อตัดสินใจเลือกสีพื้น (`fill_color`) และสีกรอบ (`stroke_color`) ที่จะใช้วาด หากค่าเป็น 0 หมายถึงช่องว่าง จะใช้สีขาว/ดำ แต่หากไม่ใช่ 0 จะไปหยิบสีจาก PALETTE ตามค่าของตำแหน่งนั้นๆ ที่เก็บใน `self.grid` นั้น แล้วเรียก `draw_square()` เพื่อวาดช่องนั้นลงบนหน้าจอจริงตามตำแหน่งและสีที่กำหนด